

FH Technikum Wien

Software Engineering 2 (SWEN)

Projektprotokoll - Zwischenabgabe

TourPlanner

Team:

Antonius Said

Milutin Kojic

Mai 2025

Version 1.1

1. Einleitung

TourPlanner ist eine webbasierte Anwendung zur Verwaltung von Touren (Wanderungen, Radtouren, Autofahrten) und dazugehörigen Tour-Logs. Die Anwendung ermöglicht das Erstellen, Bearbeiten und Löschen von Touren sowie das Erfassen von Erfahrungsberichten (Logs) mit Bewertung, Dauer, Distanz und Schwierigkeitsgrad. Das Projekt wurde im Rahmen der Lehrveranstaltung Software Engineering (SWEN) an der FH Technikum Wien entwickelt.

Ziele der Zwischenabgabe:

- Funktionsfähige REST-API mit CRUD für Touren und Tour-Logs
- Angular-Frontend mit Navigation, Formularvalidierung und Signalbasiertem Zustandsmanagement
- Implementierung von Entwurfsmustern (Strategy, Repository, Observer)
- Unit-Tests für Backend (JUnit 5 + Mockito) und Frontend (Karma/Jasmine)
- Wireframes für alle Hauptansichten

2. Technische Entscheidungen

2.1 Backend

- Spring Boot 4.0 (Java 21): Aktuelle LTS-Version, starke Community, integrierte REST-Unterstützung
- PostgreSQL 16 via Docker: Relationale Datenbank, einfache lokale Entwicklung durch Containerisierung
- Flyway: Versionierte Datenbankmigrationen, reproduzierbare Datenbankzustände
- Lombok: Reduzierung von Boilerplate-Code (Getter, Setter, Builder)
- SpringDoc OpenAPI: Automatische API-Dokumentation (Swagger UI unter /swagger-ui.html)

2.2 Frontend

- Angular 19: Aktuelle Version mit Signals-API für reaktives Zustandsmanagement
- TypeScript: Typsicherheit und bessere IDE-Unterstützung
- Bootstrap 5: Responsives Layout ohne eigenes CSS-Framework

3. Systemarchitektur

Die Anwendung folgt einer klassischen 3-Schichten-Architektur:

- Präsentationsschicht: Angular SPA (Single Page Application)
- Applikationsschicht: Spring Boot 4 REST-API
- Datenhaltungsschicht: PostgreSQL 16

3.1 Schichtendiagramm

Frontend (Angular 19): Components → Services → HTTP Client → REST API

Backend (Spring Boot 4): Controller → Service → Repository → JPA/Hibernate → PostgreSQL

3.2 REST API Uebersicht

Methode	Endpoint	Beschreibung
GET	/api/tours	Alle Touren laden
GET	/api/tours/{id}	Tour nach ID laden
POST	/api/tours	Neue Tour erstellen
PUT	/api/tours/{id}	Tour aktualisieren
DELETE	/api/tours/{id}	Tour löschen
POST	/api/tours/{id}/image	Bild hochladen
GET	/api/tours/{id}/image	Bild abrufen
GET	/api/tours/{id}/logs	Alle Logs einer Tour laden
POST	/api/tours/{id}/logs	Neuen Log erstellen
PUT	/api/tours/{id}/logs/{logId}	Log aktualisieren
DELETE	/api/tours/{id}/logs/{logId}	Log löschen

3.3 Backend-Komponenten

- TourController: REST-Endpunkte für Touren, Validierung, HTTP-Statuscodes
- TourService: Geschäftslogik, DTO-Mapping, Strategie-Aufruf
- TourRepository: Spring Data JPA, CRUD-Operationen für Touren
- TourMapper: Tour Entity <-> DTO Konvertierung
- TourLogController: REST-Endpunkte für Tour-Logs (/api/tours/{id}/logs)
- TourLogService: CRUD-Logik für Logs, Tour-Existenzprüfung
- TourLogRepository: findByTourIdOrderByDateDesc für sortierte Log-Abfrage
- TourLogMapper: TourLog Entity <-> DTO Konvertierung

3.4 Frontend-Komponenten

- AppComponent: Haupt-Layout, Sidebar-Toggle
- TourListComponent: Liste aller Touren, Selektion
- TourDetailComponent: Detailansicht mit Stats, Karten-Placeholder, Tour-Log-Liste
- TourCreateComponent: Formular für Erstellen/Bearbeiten von Touren (dual-mode)
- TourLogFormComponent: Inline-Formular für Erstellen/Bearbeiten von Logs (dual-mode)
- TourService: HTTP-Calls für Touren, Angular Signals für Zustand
- TourLogService: HTTP-Calls für Logs, Angular Signals (logs, loading, error)

4. UX-Design und Wireframes

Die Anwendung verwendet ein Split-View-Layout:

- Linke Seite (30%): Seitenleiste mit Tour-Liste und Aktionsbuttons
- Rechte Seite (70%): Detailansicht oder Formular

Wireframes wurden in draw.io erstellt (Datei: wireframes.drawio). Die Wireframes umfassen:

- Listenansicht (List-View): Seitenleiste mit Touren, leerer Hauptbereich
- Detailansicht (Detail-View): Tourdetails, Karten-Placeholder, Tour-Log-Liste
- Erstellformular (Create-Form): Formular für neue Touren mit Validierung
- Bearbeitungsformular (Edit-Form): Vorausgefülltes Formular für bestehende Touren
- Edge Cases: Leere Liste, Validierungsfehler, Ladezustand, Fehlermeldungen

4.1 Farb- und Designentscheidungen

- Primärfarbe: Dunkelblau (#2c3e50) für Sidebar
- Akzentfarbe: Gruen (#27ae60) für Aktionsbuttons
- Warnfarbe: Rot (#e74c3c) für Löschaktionen und Fehler

5. Entwurfsmuster

5.1 Strategy Pattern (Routen-Berechnung)

Zweck: Berechnung der geschätzten Reisezeit je nach Transportmittel, ohne bedingte Logik im Service.

Interface RouteCalculationStrategy:

- `getTransportType(): TransportType`
- `getAverageSpeedKmh(): double`
- `calculateEstimatedTime(double distanceKm): int` (Default-Methode: `ceil((distanz/geschwindigkeit)*60)`)

Implementierungen: CarRouteStrategy (80 km/h), BicycleRouteStrategy (15 km/h), WalkingRouteStrategy (5 km/h), RunningRouteStrategy (10 km/h)

Die RouteCalculationStrategyFactory sammelt alle Strategien per Dependency Injection und wählt die passende anhand des TransportType aus. Vorteil: Neue Transportarten können ohne Änderung bestehenden Codes hinzugefügt werden (Open/Closed Principle).

5.2 Repository Pattern

Zweck: Abstraktion des Datenzugriffs. TourRepository und TourLogRepository erweitern JpaRepository — Spring Data JPA generiert alle CRUD-Operationen automatisch.

TourLogRepository ergaenzt eine typsichere Abfragemethode: `findByTourIdOrderByDateDesc(UUID tourId)`.

5.3 Observer Pattern (Angular Signals)

Zweck: Reaktives Zustandsmanagement im Frontend. TourService und TourLogService verwenden Angular Signals (`tours`, `selectedTour`, `logs`, `loading`, `error`). Ein `effect()` in TourDetailComponent lädt automatisch die Logs wenn eine neue Tour selektiert wird — ohne manuelle Subscription-Verwaltung.

6. Unit-Tests

6.1 Backend-Tests (JUnit 5 + Mockito)

Klasse	Tests	Beschreibung
TourServiceTest	14	Repository gemockt, Mapper als Spy, Strategy Factory gemockt
TourControllerTest	15	@WebMvcTest, MockitoBean TourService, JSON-Validierung
TourLogServiceTest	11	TourLog CRUD, TourNotFoundException, TourLogNotFoundException
TourLogControllerTest	11	@WebMvcTest, Validierung Pflichtfelder, 404-Faelle
Gesamt Backend	51	

Wichtige Testfaelle TourLogServiceTest:

- findAllByTour: leere Liste / mehrere Logs / Tour nicht gefunden -> TourNotFoundException
- create: happy path gibt gespeichertes DTO zurück / Tour nicht gefunden
- update: happy path / Log nicht gefunden -> TourLogNotFoundException
- delete: ruft deleteById auf / Tour oder Log nicht gefunden -> Exception

Wichtige Testfaelle TourLogControllerTest:

- GET /logs -> 200 + JSON-Array / Tour nicht gefunden -> 404
- POST /logs -> 201 / fehlendes Datum -> 400 / Bewertung > 5 -> 400 / fehlende Schwierigkeit -> 400
- PUT /logs/{logId} -> 200 / Log nicht gefunden -> 404
- DELETE /logs/{logId} -> 204 / Log nicht gefunden -> 404

6.2 Frontend-Tests (Karma/Jasmine)

Klasse	Tests	Beschreibung
TourService (Angular)	20	HttpTestingController, Signal-Updates nach allen CRUD-Ops
TourCreateComponent	20	3 describe-Blöcke: Create, Edit (kein Cache), Edit (Cache)
TourLogService (Angular)	10	HttpTestingController, Signals: prepend/replace/remove
TourLogFormComponent	17	Formularvalidierung, Create/Edit-Modus, saved/cancelled Events
AppComponent	3	Initialisierung, Sidebar-Toggle
Gesamt Frontend	70	

Klasse	Tests	Beschreibung
Gesamt gesamt	121	

7. Time-Tracking

Datum	Wer	Aktivitaet	Std.
15.03.2025	KOJIC	Projektsetup, Spring Boot Initialisierung	3,0
15.03.2025	KOJIC	Angular-Projekt Setup, Routing konfiguriert	2,5
22.03.2025	KOJIC	Datenbankmodell, Flyway-Migrationen	3,0
22.03.2025	KOJIC	Angular-Komponenten-Struktur, Services	3,0
29.03.2025	KOJIC	TourController, TourService implementiert	4,0
29.03.2025	KOJIC	TourListComponent, TourDetailComponent	3,5
05.04.2025	SAID	Strategy Pattern implementiert	2,0
05.04.2025	KOJIC	TourCreateComponent (Create/Edit-Mode)	4,0
12.04.2025	SAID	TourLog Backend (Entity, Service, Controller)	3,5
12.04.2025	SAID	TourLog Frontend (Service, UI-Komponenten)	3,5
19.04.2025	SAID	Backend Unit-Tests (JUnit 5 + Mockito)	3,5
19.04.2025	SAID	Frontend Unit-Tests (Karma/Jasmine)	3,0
26.04.2025	KOJIC	Bild-Upload, Bugfixes, Code-Review	2,5
26.04.2025	SAID	Frontend-Design, Wireframes, Protokoll	3,0
Gesamt			44,0

8. Ausblick

8.1 Aktueller Stand

- REST-API vollständig implementiert (CRUD für Touren und Tour-Logs, Bild-Upload)
- Angular-Frontend funktionsfähig mit Formularvalidierung und Tour-Log-Verwaltung
- Entwurfsmuster implementiert (Strategy, Repository, Observer)
- Unit-Tests: 51 Backend-Tests (JUnit 5) + 70 Frontend-Tests (Karma/Jasmine) = 121
gesamt

8.2 Noch zu implementieren (Finalabgabe)

- Kartenintegration (OpenRouteService oder OpenLayers)
- PDF-Export für Touren und Tourlogs
- Verbesserte Fehlerbehandlung im Frontend
- End-to-End Tests (Cypress oder Playwright)
- Docker-Compose für gesamte Anwendung

9. Quellenverzeichnis

Spring Boot Dokumentation: <https://docs.spring.io/spring-boot/>

Angular Dokumentation: <https://angular.dev/>

Spring Data JPA: <https://docs.spring.io/spring-data/jpa/>

Flyway Dokumentation: <https://documentation.red-gate.com/flyway>

OpenAPI/SpringDoc: <https://springdoc.org/>

Karma Test Runner: <https://karma-runner.github.io/>